1. Overzicht van de programmastructuur

De PLC-code in TIA Portal is opgebouwd rond een hiermee 10 ms cyclisch uitgevoerd organisatieblok (OB35). Dit blok fungeert als orkestrator: het leest sensordata in, initialiseert PID-parameters, roept de regelfunctieblokken aan en schrijft de resulterende commando's terug naar de actuatoren.

Blok	Type	Functie
OB35	OB (cyclisch, 10 ms)	Hoofdcontrolecyclus — orkestrator
FB_PID_M0 / M1 / M2	FB	Generieke PID-regelaar (P, I, D met anti-windup)
FB_M1_HeightComp	FB	Hoogteregelaar Motor 1 (FF + FB)
FB_M2_AngleComp	FB	Hoekcompensatie Motor 2 (FF + FB)
FB_PathFollower	FB	Toestandsmachine — bepaalt setpoints
FB_Safety	FB	E-stop, eindschakelaars, sensor-watchdog
DB_GlobalParams	DB	Afstelbare PID-parameters en doelwaarden
DB_SensorActual	DB	Actuele sensordata en uitvoercommando's
DB_PIDState	DB	Interne PID-toestand (Kp, Ti, Td, integraal…)

2. OB35 — de 10 ms cyclische orkestrator

OB35 is het startpunt van elke regelcyclus. De hardware-timer van de CPU triggert dit blok exact elke 10 milliseconden. De structuur bestaat uit de volgende netwerken:

2.1 Network 1 — Initialisatie (eerste cyclus)

Bij de allereerste uitvoering (vlag ‘eerste_cyclus’) worden alle PID-parameters vanuit DB_GlobalParams overgeschreven naar DB_PIDState:

```
DB_PIDState.pid_M1.Kp  <- DB_GlobalParams.M1_Kp
DB_PIDState.pid_M1.Ti_s <- DB_GlobalParams.M1_Ti_s
DB_PIDState.pid_M1.Td_s <- DB_GlobalParams.M1_Td_s
DB_PIDState.pid_M1.dt_s <- 0.01   (10 ms scantime)
DB_PIDState.pid_M1.deadband <- DB_GlobalParams.M1_deadband_mm
```

Dit patroon geldt ook voor M2 en M0. Het voordeel is dat parameters online kunnen worden aangepast in DB_GlobalParams zonder het programma opnieuw te compileren of te downloaden.

2.2 Network 5 — Inclinatiemeting filteren

De inclinatiemeting van M0 wordt via een eerste-orde laagdoorlaatfilter verwerkt. Dit reduceert meetruis en vermijdt schokken in de regeluitgang:

```
incl_gefilterd = alfa * incl_prev + (1 - alfa) * incl_gemeten
```

De **alfa_inclinatie**-parameter in DB_GlobalParams bepaalt de filtersterkte. Een waarde dicht bij 1 geeft sterk filteren (traag), dicht bij 0 geeft weinig filtering (snel).

2.3 Network 6 — Veiligheid (FB_Safety)

Voor elk regelblok wordt FB_Safety aangeroepen. Dit blok controleert:

- Hardware e-stop (schakelaar op de machine)
- Eindschakelaars boven/onder voor M1 en plus/min voor M2
- Sensor-watchdog: als de timestamp van de sensordata niet wijzigt binnen een referentievenster, wordt een communicatiefout gegenereerd

De uitgang ‘veilig’ (boolean) is het poortje voor alle verdere regelacties. Als veilig = FALSE, worden enable-signalen geblokkeerd.

2.4 Network 8 — Padvolger (FB_PathFollower)

De toestandsmachine FB_PathFollower ontvangt de gewenste doel-positie en -hoek uit DB_GlobalParams en beslist op basis van de actuele sensorpositie:

- Of M1 en M2 mogen bewegen (enable-signalen)
- Wat de doelsetpoints zijn (M1_target_mm, M2_target_deg)
- Of het doel bereikt is (doel_bereikt vlag)

De toestandsmachine houdt ook rekening met M0 (translatiemotor): via M0_max_snelheid en M0_rem_afstand wordt het rijprofiel bepaald.

2.5 Network 10 en 14 — Regelblokken M1 en M2

De eigenlijke feedforward + feedback regelingen worden hier uitgevoerd. De sensordata worden doorgegeven als proceswaarden, de doelwaarden als setpoints. De resulterende commando’s worden opgeslagen in DB_SensorActual.cmd.

3. Feedforward

Feedforward is een regelstrategie waarbij de te verwachten verstoring op voorhand berekend wordt, zonder te wachten op een fout. De code bevat twee afzonderlijke feedforward-berekeningen: één voor de hoogte van M1 en één voor de hoek van M2.

> **Principe van feedforward**
>
> Als de machine kantelt door de inclinatie van de ondergrond, verschuift het eindpunt geometrisch.
>
> Met feedforward berekent de PLC die verschuiving op basis van meetwaarden en trigonometrie,
>
> en compenseert proactief — nog voor de PID-lus een fout kan detecteren.

3.1 FB_M1_HeightComp — Feedforward hoogte (Networks 1 t/m 4)

De feedforward voor M1 berekent hoeveel de positie van het eindpunt verticaal verschuift door de inclinatie van de basis. De berekening verloopt in vier stappen:

Network 1 — Graden naar radialen

```
theta_rad = inclinatie_deg * PI / 180
```

De inclinatiehoek (gemeten in graden) wordt omgezet naar radialen, nodig voor de trigonometrische functies.

Network 2 — Verticale basisverplaatsing

```
delta_z_basis_mm = d_pivot_m * sin(theta_rad) * 1000
```

De pivotafstand **d_pivot_m** (afstand van draaipunt tot eindpunt, in meter) vermenigvuldigd met de sinus van de hoek geeft de verticale verplaatsing van het basispunt. Vermenigvuldigd met 1000 om van meters naar millimeters te gaan.

Network 3 — Verplaatsing van M1 zelf

```
delta_z_M1_mm = (h_M1_nominaal_mm / 1000) * (1 - cos(theta_rad)) * 1000
```

Door de rotatie verandert ook de hoogtebijdrage van M1 zelf. De nominale hoogte **h_M1_nominaal_mm** wordt gecorrigeerd via de cosinus van de hoek. Dit is het ‘verlies’ aan hoogte door de rotatie.

Network 4 — Totale feedforward

```
FF_mm = delta_z_basis_mm - delta_z_M1_mm
```

Het verschil van beide verplaatsingen geeft de netto feedforward-correctie in millimeters. Dit is de geschatte verstoring die de PID-lus anders zelf zou moeten corrigeren.

3.2 FB_M2_AngleComp — Feedforward hoek (Networks 2 t/m 3)

De feedforward voor M2 is complexer omdat twee verstoringen spelen: de laterale verschuiving door roll en de longitudinale verschuiving door pitch.

Network 2 — Laterale fout en FF_lat

```
roll_rad = inclinatie_roll_deg * PI / 180
lat_err_m = radar_eff * tan(roll_rad)
FF_lat_deg = -atan(lat_err_m / radar_eff) * 180 / PI
```

De effectieve radarafstand **radar_eff** (minimaal 0.1 m) wordt gebruikt om de laterale verplaatsing te berekenen. Via de boogtan functie wordt dit omgezet naar een hoekcompensatie in graden.

Network 3 — Pitch-compensatie en totale FF

```
FF_pitch_deg = -pitch_rad * (L_arm_m / radar_eff) * 180/PI * 0.5
FF_totaal_deg = FF_lat_deg + FF_pitch_deg
```

De armlengte **L_arm_m** bepaalt de geometrische invloed van de pitch op de eindpositie. De factor 0.5 is een empirische schaalfactor. De twee feedforward-bijdragen worden opgeteld tot de totale FF voor M2.

4. Feedback — de PID-regelaar (FB_PID_M0)

De generieke PID-engine FB_PID_M0 wordt als aparte instantie gebruikt voor zowel M1 (als FB_PID_M1) als M2 (als FB_PID_M2). Elke instantie heeft zijn eigen DB en eigen parameters via DB_PIDState. De volgende netwerken implementeren de klassieke PID-structuur met enkele uitbreidingen:

4.1 Network 1 — Reset

Bij een resetcommando worden de integrator, vorige fout en gefilterde D-term op nul gezet. Dit voorkomt een schok bij het opnieuw inschakelen van de regelaar.

4.2 Network 5 — Foutberekening (feedback-kern)

```
e_huidig = setpoint - proceswaarde
```

Dit is de **feedbacklus**: de werkelijke meting (**proceswaarde**) wordt afgetrokken van het gewenste setpoint. De fout **e_huidig** is de drijvende kracht achter alle PID-berekeningen.

4.3 Networks 6-7 — Deadband

Als **|e| ≤ deadband**, wordt **in_deadband = TRUE** gezet. De P- en D-term worden dan geblokkeerd. Dit vermijdt constante kleine oscillaties wanneer het systeem dicht bij het setpoint zit (hysterese-effect).

4.4 Network 8 — Proportionele term

```
P_term = Kp * e_huidig
```

Directe versterking van de fout. Een grote fout geeft een grote correctie-impuls.

4.5 Network 10 — Integratieve term met anti-windup

```
I_term = I_prev + (Kp / Ti_s) * e_huidig * dt_s + (1 / Tt_s) * track_err *
dt_s
```

De integrator telt de fout op over de tijd, waardoor ook een kleine, aanhoudende fout uiteindelijk weggeregeld wordt. Twee bijzonderheden:

- Anti-windup via tracking: track_err is het verschil tussen de ongeclampte uitgang (u_raw) en de verzadigde uitgang (u_sat). Zodra de actuator op zijn grens zit, stuurt track_err de integrator terug — zo ‘windt’ de integrator niet verder op.
- Ti_s > 0 check: als Ti_s nul of negatief is, wordt de integratieterm overgeslagen (integrator uitgeschakeld).

4.6 Networks 12-13 — Differentiatieve term met filter

```
alfa_D = Td_s / (Td_s + 10 * dt_s)
D_term = alfa_D * D_prev + Kd * (1 - alfa_D) * (e_huidig - e_prev)
```

De ruwe afgeleidde **(e - e_prev) / dt** is gevoelig voor meetruis. De code past een eerste-orde filter toe met de factor **alfa_D**. Bij een kleine alfa_D is het filter sterk (traag), bij een alfa dicht bij 1 is het filter zwak (snel). **Kd = Kp * Td_s / (10 * dt_s)**.

4.7 Network 14 — Optelling P + I + D

```
u_raw = P_term + I_term + D_term
```

De drie termen worden opgeteld tot de ruwe regeluitgang.

4.8 Network 15 — Saturatie en clamping

```
als u_raw < out_min:  u_sat = out_min, saturatie = TRUE
als u_raw > out_max:  u_sat = out_max, saturatie = TRUE
anders:               u_sat = u_raw,  saturatie = FALSE
```

De uitgang wordt begrensd tot het fysisch haalbare bereik **[out_min, out_max]**. De vlag **saturatie** signaleert dat de actuator op zijn grens zit. Dit signaal is ook de trigger voor de anti-windup terugkoppeling naar de integrator.

4.9 Network 16 — Geheugenupdate

Aan het einde van elke cyclus worden e_huidig en u_sat opgeslagen als e_prev en u_raw_prev. Deze worden gebruikt in de volgende cyclus voor de D-term en de anti-windup berekening.

5. Optelling feedforward + feedback

De feedforward en de PID-uitgang worden gecombineerd in Network 7 van respectievelijk FB_M1_HeightComp en FB_M2_AngleComp.

5.1 FB_M1_HeightComp — Network 7

```
SP_totaal = doel_hoogte_mm + FF_mm + (PID_uitgang - doel_hoogte_mm)
          = FF_mm + PID_uitgang
```

De feedforward verschuift het setpoint proactief. De PID-uitgang corrigeert de resterende fout. De som is het uiteindelijke actuatorcommando M1_setpoint_mm.

De code splitst de bijdragen expliciet op voor logging en analyse:

- FF_bijdrage_mm = FF_mm (puur geometrisch berekend)
- FB_bijdrage_mm = PID_uitgang - doel_hoogte_mm (puur de correctie van de PID)

5.2 FB_M2_AngleComp — Network 6

```
SP_totaal = FF_totaal_deg + pid_inst_M2.uitgang
```

Identiek principe als bij M1: de feedforward (FF_lat + FF_pitch) telt bij de PID-correctie op tot het totale hoeksetpoint voor M2.

5.3 Begrenzing uitgang (Networks 8-9)

```
als SP_totaal < out_min:  SP_totaal = out_min
als SP_totaal > out_max:  SP_totaal = out_max
```

Na optelling wordt het gecombineerde setpoint nogmaals begrensd. Dit vangt extreme waarden op die zouden kunnen ontstaan bij een grote feedforward-bijdrage gecombineerd met een grote PID-correctie.

6. Samenvatting regelstructuur

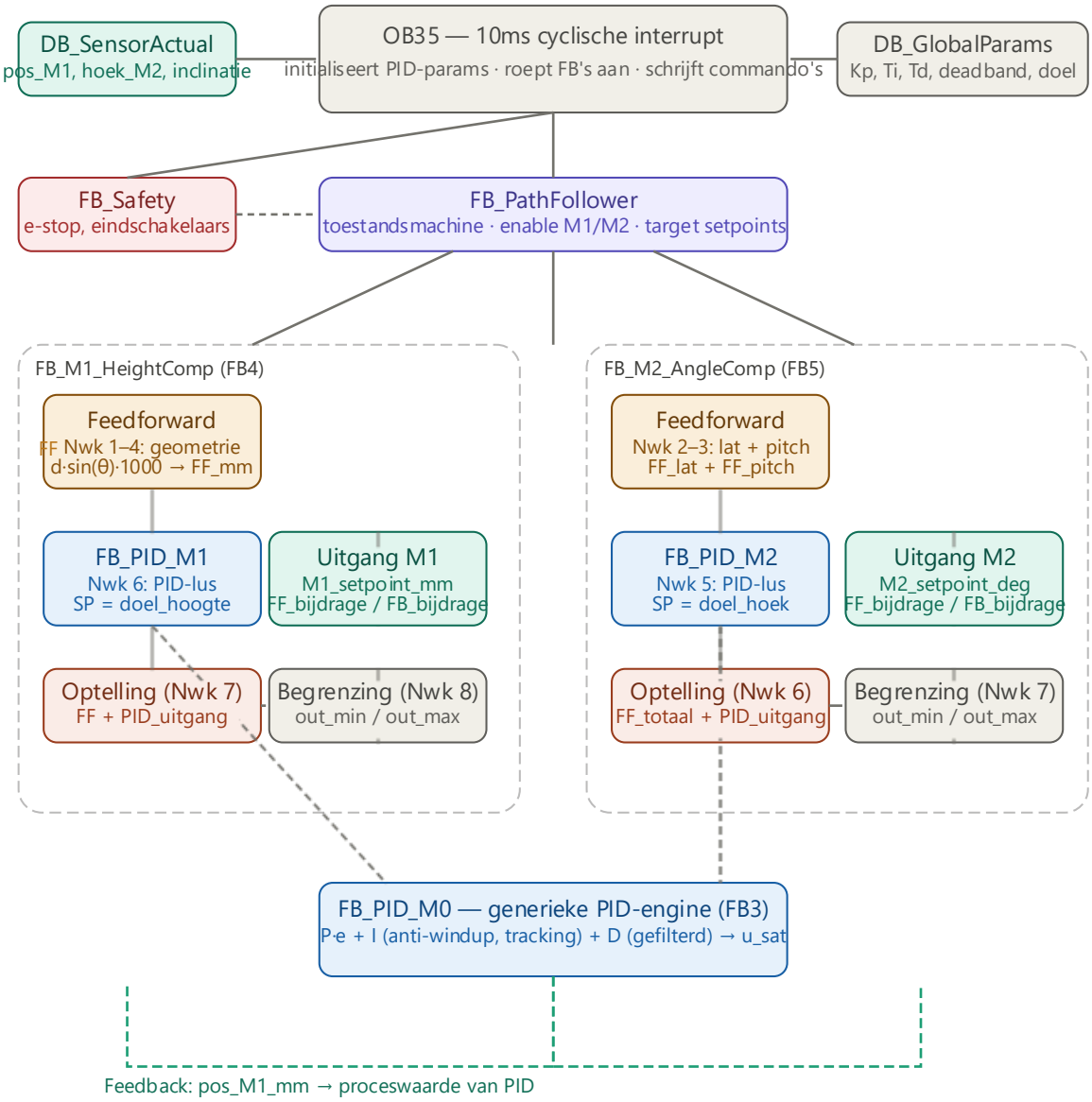
De code implementeert een klassieke feedforward + feedback combinatie per motor. Onderstaande tabel vat de rolverdeling samen:

Component	Rol	Locatie in code
Feedforward M1	Geometrische compensatie inclinatie (basis + M1)	FB_M1_HeightComp, Nwk 1-4
Feedforward M2	Laterale + pitch-compensatie via radarafstand	FB_M2_AngleComp, Nwk 2-3
PID M1 (feedback)	Corrigeert resterende hoogte-fout	FB_PID_M1, alle netwerken
PID M2 (feedback)	Corrigeert resterende hoek-fout	FB_PID_M2, alle netwerken
Deadband	Voorkomt oscillatie dicht bij setpoint	FB_PID_M0, Nwk 6-7
Anti-windup	Voorkomt integrator-overloop bij saturatie	FB_PID_M0, Nwk 10 (track_err)
D-filter	Ruisonderdrukking op de D-term	FB_PID_M0, Nwk 12-13
Begrenzing	Fysische limiet actuatoruitgang	FB_PID_M0 Nwk 15, HeightComp Nwk 8-9
Veiligheid	E-stop, eindschakelaars, watchdog	FB_Safety, OB35 Nwk 6

Voordeel van de feedforward + feedback combinatie

De feedforward reageert instant op een meetbare verstoring (inclinatie, roll, pitch), zonder vertraging van een integrator. De PID-lus corrigeert wat de feedforward niet perfect wegwerkt: modelfout, slijtage, niet-gemodelleerde verstoringen. Samen geven ze een snelle en nauwkeurige regelaar die robuust is voor zowel bekende als onbekende verstoringen.

Regelarchitectuur — OB35 elke 10 ms



Legenda

- Feedforward (geometrisch berekend)
- Sensormeting → proceswaarde
- Opgetelde uitgang (FF + FB)
- Feedback / PID-correctie
- Orkestrator-aanroep (OB35 → FB)
- Terugkoppelingslus